DB Monitoring: стратегия деплоя и отката

Домашнее задание B10 «Стратегии деплоя и релиз-процессы» (слайд 45; практика — слайды 33–38), Issue #14. Сверено с `master` `f891234` на 10.11.2026. Это проектное решение: переключатель трафика, автоматическое сравнение версий и испытание отката не реализованы и не проводились; «100 % трафика» — целевое состояние схемы.

1. Что выпускается

Приложение — Flask со встроенным планировщиком: сбор метрик (micro-batch, по умолчанию раз в 15 мин), оценка аномалий, уведомления, ночное обучение. Compose поднимает PostgreSQL-источник, TimescaleDB-хранилище, Flask и по профилю `inference` FastAPI-инференс без состояния (реестр только для чтения). Выпускаются два независимых артефакта: **образ приложения** (тег образа `X.Y.Z`; `/healthz.version` показывает `vX.Y.Z` из `APP_VERSION`) и **версия модели** Isolation Forest в реестре (`if-<SHA-256>`, схема признаков `1.0.0`). Версия модели включается для каждого источника отдельно — привязкой (`bindings`) адаптера. Основной поток работы — тики сборщика, а не HTTP-запросы.

2. Выбор: blue-green для приложения, shadow → canary для модели

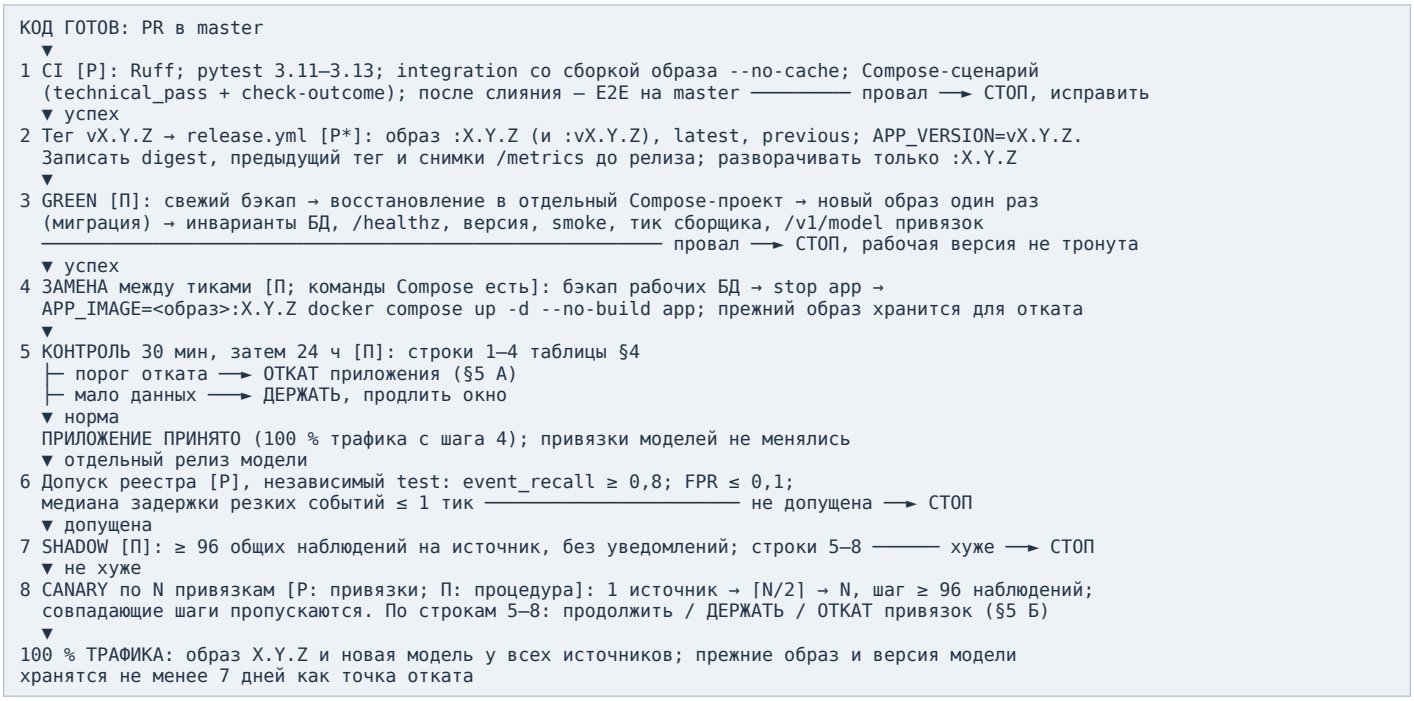
Приложение — blue-green как целевой подход; на стенде — проверка green и последовательная замена. Ошибка кода проявляется сразу (`/healthz`, 5xx, сбой сборщика), поэтому важнее быстро вернуть рабочую версию, чем постепенно набирать уверенность (вопрос слайда 38). Классический blue-green требует двух готовых сред и переключателя трафика. На стенде их нет, и вторую среду Flask нельзя держать запущенной: планировщик стартует в каждом процессе без отключающей настройки (двойной сбор и обучение), а миграции хранилища требуют остановить старые процессы (migrations.md). Доступный способ: green проверяется в изолированном Compose-проекте на копии данных, затем рабочий контейнер заменяется между тиками с простоем порядка минуты (оценка). Прежний образ — артефакт для отката, а не работающая среда: возврат — повторная замена, а не мгновенное переключение.

Модель — shadow, затем canary по источникам. Ошибка модели «тихая» (слайд 12): сервис работает, но хуже находит инциденты. Нужны проверка на реальных данных и сравнение с предыдущей версией. Основа есть: версия задаётся для каждого источника, оценки хранятся с ключом версии, откат — возврат привязки без переобучения. Единица «трафика» — источник (подключение + схема + таблица).

Отклонено. Canary приложения: нет маршрутизатора, у внутреннего инструмента мало пользователей, две версии делили бы хранилище и фоновые задания. Blue-green модели «сразу на все источники»: быстрый откат уже дают привязки, а одновременное переключение скрывает ухудшение до пропущенных инцидентов.

3. Схема процесса: от «код готов» до «100 % трафика»

[P] — реализовано в репозитории; [P*] — реализовано, но в этом репозитории не запускалось; [П] — предлагается.



Реализовано и проверялось (CI, тесты, протоколы)	Есть в коде, но здесь не запускалось	Нет — предлагается этим документом
CI `tests.yml`/`lint.yml` и Compose-сценарий #13; реестр и допуск `admitted()` #5; привязки и оценки с ключом версии #12; `/healthz`, `/metrics`, журнал `collector_runs`, алерты `HttpErrorRatioHigh` #15 и `ModelAnomalyRateHigh` #16; backup/restore (тесты)	`release.yml`, `promote.yml` — тегов `v*.*.*` нет; runbook отката — dry-run не зафиксирован; процедура миграций на рабочих данных	Две обслуживающие среды и переключатель трафика; фоновая shadow-оценка (ручная оценка новейшей точки неактивной версией подавляет её уведомление) и сравнение версий; Prometheus-сервер; допущенная модель (её нет, §6); внешняя доставка алертов

4. Критерии успешности и пороги

Новая версия сравнивается с предыдущей: приложение — с окном той же длины до релиза, модель — на тех же наблюдениях. Счётчики `/metrics` обнуляются при замене, Prometheus-сервера нет: «до релиза» — разность двух снимков `/metrics` за 30 мин до замены из записи релиза. Правила #15 (10 %, ≥ 10 запросов) и #16 (k из 20) есть в коде, остальные пороги **проектные**. Неполное окно — всегда «держать».

№	Показатель и источник	Единица, окно	Продолжить	Держать, разобрать	Откат
1	Доступность: `/healthz`, Docker HEALTHCHECK раз в 30 с	доля успешных проб; 30 мин	100 %, `version` = `vX.Y.Z`	1–2 неуспешные пробы	3 подряд (`unhealthy`) или новый `down` компонента
2	HTTP 5xx: `http_requests_total` без `/metrics`	%; 5 мин, ≥ 10 запросов	≤ 1 % и ≤ доли до релиза + 1 п. п.	> 1 % и ≤ 10 %	> 10 % (`HttpErrorRatioHigh`) или 5xx на smoke
3	Время ответа p95: `http_request_duration_seconds`	с; 15 мин, ≥ 30 запросов	≤ 1,2 × p95 до релиза	> 1,2 ×	> 2 × или > 2 с два окна подряд
4	Сбор: журнал `collector_runs` по подключениям (счётчик — без разбивки)	плановые запуски; 2 тика	`success`/`warning` у всех подключений, успешных до релиза	один `failed` или пропуск тика	2 `failed` подряд у подключения, успешного до релиза
5	Ошибки инференса: `inference_predictions_total`, `outcome` ≠ `ok`	% вызовов; до ≥ 20 вызовов (1 источник ≈ 5 ч)	≤ 1 %	> 1 % и ≤ 5 %	> 5 % или `model_unavailable` новой версии
6	Качество и свежесть входов: `inputs.conclusive` (#16)	да / нет; ≥ 20 валидных векторов	`true`	`false`	сбой сбора после релиза — строка 4
7	Доля аномалий против baseline той же версии: `ModelAnomalyRateHigh`	аномалий среди 20 последних	< k из baseline этой версии (≥ 500 норм. наблюдений); 3 из 20 — только demo #13	≥ k или нет baseline	только вместе со строкой 8
8	Расхождение с предыдущей версией на тех же наблюдениях	наблюдения; ≥ 96 на источник (≈ 24 ч)	≤ 2 новых флагов; флаги на подтверждённых инцидентах сохранены	> 2 или не разобраны	пропущен подтверждённый инцидент; firing строки 7 только у новой версии

Обоснование: 1 % — в норме 5xx около 0 (#15); 1,2 × — запас на шум и корзины гистограммы; 96 наблюдений — сутки сбора; «новый флаг» — аномалия у новой версии при норме у старой, 2 из 96 — порядок FPR demo-модели на test (1,66 %). Precision/recall/FPR — только на размеченном test (этап 6), а не по потоку.

5. План отката

Запуск и решение. Условие столбца «Откат» или порча данных. Приложение откатывает ответственный за релиз (DevOps, M4 MLOps-roadmap), модель — ML-инженер; инциденты строк 7–8 подтверждает инженер данных.

А. Приложение. Цель восстановления обслуживания — ≤ 10 мин, с восстановлением БД ≤ 2 ч (плановые, не измерены). (1) Зафиксировать время, `/healthz.version`, снимок `/metrics`, журналы `app`. (2) Функция под флагом (`FF_FORECAST`, `FF_LLM_NOTIFICATIONS`): изменить `.env` (в контейнер — через `env_file`) и пересоздать контейнер с тем же образом: `APP_IMAGE=<образ>:X.Y.Z docker compose up -d --no-build --force-recreate app`. `restart` окружение не перечитывает, а без явного `APP_IMAGE` Compose возьмёт образ из `.env`. Проверить `/healthz` (200, `vX.Y.Z`) и `/api/forecast/…` → 404; не помогло — шаг 3. (3) `APP_IMAGE=<образ>:<предыдущий X.Y.Z> docker compose up -d --no-build app` — тег с этапа 2, не плавающий `previous`. (4) Схема хранилища изменена — старый код на неё не запускать. `MONITOR_DB_URL` задан в `environment` Compose и перекрывает `.env`, поэтому DSN не меняют: бэкап восстанавливают в новую БД того же TimescaleDB, проверяют прежним образом в изолированном проекте (migrations.md) и при остановленном `app` переименовывают базы (`metrics` → `metrics_failed`, копия → `metrics`). Данные после бэкапа — только отдельным решением.

Б. Модель (цель ≤ 1 тик, 15 мин). Вернуть в `bindings` прежние `model_version` и `feature_schema_version`; по коду файл читается при каждом обращении, перезапуск не нужен (не испытано); версии реестра не удаляются. Реестр сверяет схему признаков и scikit-learn и отказывает без fit (`model_unavailable`): при откате образа сначала проверить `/v1/model` прежней версии.

В. Обработанные запросы и результаты. Запросы во время замены получают ошибку и повторяются; пропущенный тик не заполняется. Оценки хранятся по ключу «источник + время + версия + схема» (`ON CONFLICT DO NOTHING`): результаты новой версии остаются для разбора, дашборд читает прежнюю пару, она досчитывает недостающие точки без повторных уведомлений. Отправленные уведомления не отзываются — их список вносится в запись об инциденте; собранные метрики не удаляются.

Г. Подтверждение. Обслуживание восстановлено (в пределах цели ≤ 10 мин): контейнер `healthy`; `/healthz` 200 с прежней `vX.Y.Z` 5 проб подряд; smoke без 5xx; в первом окне 5 мин с ≥ 10 запросами (иначе — smoke-запросы) условие `HttpErrorRatioHigh` ложно. Затем наблюдение вне цели: 2 тика без `failed`, `outcome=ok` у всех привязок, `resolved` — только для алертов, бывших в `firing`.

6. Ограничения

- `release.yml` не запускался (тегов `v*.*.*` нет). Runbook и README: образ `…/db-monitoring` вместо `ghcr.io/<владелец>/<репозиторий>`; `docker pull …:previous && docker compose up -d app` без `APP_IMAGE` образ не сменит; `restart` не применяет `.env`; `/healthz?strict=true` при rate-limit `memory://` по коду всегда 503. Правка runbook — отдельная задача.
- Допущенной модели нет (#9: FPR 13,27 %; demo #13 не допущена): этап 6 сейчас — «СТОП», точки отката модели нет.
- Откат не испытывался. Legacy IF без привязки перезаписывается ночью без версий — откат образа его не вернёт.